

Hybrid Parallel Data Structure: Balancing Strong and Local Linearizability

<https://github.com/levleyfer/parallelProgrammingproject.git>

By Daniel Fraimovich 315880963

Lev Leyfer 321385064

Abstract

Concurrency in distributed systems often needs a delicate trade-off between performance and consistency. Linearizability, which is the usual correctness rule for concurrent objects, makes sure operations look like they happen instantly and follow real-time order. But strict linearizability can slow performance a lot. Two well-known alternatives are **strong linearizability** and **local linearizability**:

Strong Linearizability tries to keep a stricter order of operations than usual linearizability. This can provide better guarantees for some applications, but it might add cost.

- Ensures a strict ordering of operations, maintaining a real-time sequence where every operation appears to take effect instantaneously.
- In your system, the global counter follows strong linearizability.

Local Linearizability lets different parts of the system handle their own operations in a more relaxed way, which might give better performance but can weaken the global consistency guarantees.

- Allows different parts (shards) to be updated independently without requiring strict global ordering.
- This improves performance by reducing synchronization overhead.
- The local data stored in shards follows local linearizability.

This paper summarizes and synthesizes two academic works:

1. ["Strong Linearizability using Primitives with Consensus Number 2" \(Attiya, Castaneda, Enea, 2024\)](#)
2. ["Local Linearizability" \(Haas et al., 2015\)](#)

Both works explore ways to enhance performance while maintaining acceptable levels of consistency. In this paper we will propose a hybrid parallel data structure that selectively applies strong and local linearizability, aiming to optimize both correctness and efficiency.

Summary of Articles

Local Linearizability

This paper introduces the concept of *local linearizability*, a weaker consistency condition than traditional linearizability. The central claim is that, by allowing different parts (or components) of a concurrent data structure to be linearized independently, local linearizability reduces the global synchronization overhead and enables more scalable implementations.

To support this claim, the authors present both theoretical arguments—showing that local linearizability still preserves key correctness guarantees—and experimental results demonstrating efficient, high-performing implementations of locally linearizable queues and stacks.

They show that by splitting the system into parts (using a decomposition principle), Each part's ordering can be checked on its own, making the overall approach simpler for system designers to understand and verify.

In the article, the implementations of locally linearizable data structures (like k-FIFO Queue and k-Stack) involve dividing the data structure into smaller, independent units that can be operated on concurrently by different threads. These units are referred to as **segments** (in k-FIFO and k-Stack) or **backends** (in Distributed Queues and Stacks).

a. **k-FIFO Queues and k-Stacks**

In the k-FIFO Queue and k-Stack implementations, the data structure is organized as a list of segments, where each segment is an array-like structure with k slots to hold elements. Segments allow multiple threads to insert and remove elements concurrently, with each segment acting as a semi-independent unit.

b. **Backends in Distributed Queues and Stacks**

In the Distributed Queues and Distributed Stacks implementations, the data structure is a segment (a single array) containing p backends, where each backend is itself a linearizable queue.

Each backend is assigned to threads either statically or dynamically (one per thread). Threads primarily insert into their own backend but can remove from any backend, ensuring local linearizability by preserving thread-local order while allowing global flexibility.

Implementation of the Local Linearizability

K-FIFO Queue -Explanation behind the Class:

Focusing on their purpose and how they contribute to local linearizability.

Constructor(n, kValue)

Purpose: Initializes the k-FIFO queue with a fixed number of threads (n) and slots per segment (k).

How It Works: Creates an array of Segment objects, one per thread, each with k slots. This ensures each thread has its own local storage, supporting local linearizability by isolating operations.

Key Detail: The separation into segments means each thread's operations are independent, reducing contention.

enqueue(item)

Purpose: Adds an item to the queue in the calling thread's segment.

How It Works:

Gets the current thread's ID and its segment.

In a loop, check the tail position. If there's space ($\text{tail} < k$), it attempts to insert the item using a Compare-And-Swap (CAS) operation.

If CAS succeeds, it advances the tail and returns true. If the segment is full, it returns false (or could implement overflow logic).

Concurrency: The CAS ensures atomic insertion, and operating on a thread-local segment avoids inter-thread interference.

Local Linearizability: Ensures the thread's enqueue operations appear sequential within its own history.

dequeue()

Purpose: Removes and returns an item from the thread's segment.

How It Works:

Accesses the thread's segment and checks the head position against tail.

If there's an item ($\text{head} < \text{tail}$), it retrieves the item and uses CAS to nullify the slot.

If CAS succeeds, it advances head and returns the item. If empty, returns null (or could steal from another segment).

Concurrency: CAS ensures atomic removal, maintaining consistency within the thread's segment.

Local Linearizability: Dequeue operations are consistent with the thread's own enqueue operations.

```
Class LocallyLinearizableKFIFOQueue {
    Segment[] segments; // Array of thread-local segments
    int k; // Number of FIFO slots per segment
    int numThreads; // Number of threads
    Constructor(n, kValue) {
        numThreads = n;
        k = kValue;
        segments = new Segment[numThreads];
        for i = 0 to numThreads - 1 {
            segments[i] = new Segment(k);
        }
    }
    Function enqueue(item) {
        threadId = getThreadId();
        segment = segments[threadId];
        while true {
            tail = segment.getTail();
            if tail < k {
```

```

        if segment.compareAndSet(tail, item, null) {
            segment.advanceTail(tail);
            return true;
        }
    } else {
        // Segment full, handle overflow (e.g., retry or move)
        return false;
    }
}

}

Function dequeue() {
    threadId = getThreadId();
    segment = segments[threadId];
    while true {
        head = segment.getHead();
        if head < segment.getTail() {
            item = segment.getItem(head);
            if segment.compareAndSet(head, null, item) {
                segment.advanceHead(head);
                return item;
            }
        } else {
            // Segment empty, handle underflow (e.g., steal or wait)
            return null;
        }
    }
}
}
}

```

Segment - Explanation behind the Class:

Constructor(size)

Purpose: Sets up a segment with size (i.e., k) slots.

How It Works: Initializes an array of atomic references (for thread-safe updates) and atomic integers for head and tail.

Key Detail: Atomic variables enable safe concurrent access, though in this design, each segment is typically thread-local.

getTail()

Purpose: Returns the current tail position (next free slot).**How It Works:** Simply reads the tail atomic integer.

Role: Used by enqueue to check for space.

getHead()

Purpose: Returns the current head position (next item to dequeue).

How It Works: Reads the head atomic integer.

Role: Used by dequeue to locate the next item.

advanceTail(oldTail)

Purpose: Moves the tail pointer forward after a successful enqueue.

How It Works: Uses CAS to update tail from oldTail to oldTail + 1, ensuring no concurrent modification.

Concurrency: Prevents race conditions if multiple operations occur (though typically thread-local).

advanceHead(oldHead)

Purpose: Advances the head pointer after a successful dequeue.

How It Works: Similar to advanceTail, uses CAS to increment head.

Concurrency: Ensures atomic updates.

compareAndSet(index, expected, newValue)

Purpose: Atomically updates a slot if it matches the expected value.

How It Works: Uses the atomic reference at slots[index] to perform a CAS operation.

Role: Core mechanism for thread-safe enqueue (set item) and dequeue (clear slot).

getItem(index)

Purpose: Retrieves the item at a specific slot.

How It Works: Returns the value stored in slots[index].

Role: Used by dequeue to fetch the item before removal.

```
Class Segment {  
    AtomicReference[] slots;    // Array of slots with atomic references  
    AtomicInteger head;        // Points to next item to dequeue  
    AtomicInteger tail;        // Points to next free slot  
}
```

```

Constructor(size) {
    slots = new AtomicReference[size];
    for i = 0 to size - 1 {
        slots[i] = new AtomicReference(null);
    }
    head = new AtomicInteger(0);
    tail = new AtomicInteger(0);
}

Function getTail() {
    return tail.get();
}

Function getHead() {
    return head.get();
}

Function advanceTail(oldTail) {
    tail.compareAndSet(oldTail, oldTail + 1);
}

Function advanceHead(oldHead) {
    head.compareAndSet(oldHead, oldHead + 1);
}

Function compareAndSet(index, expected, newValue) {
    return slots[index].compareAndSet(expected, newValue);
}

Function getItem(index) {
    return slots[index].get();
}
}

```

Properties and Theoretical Contributions

Compositionality:

Local linearizability is compositional: a history over multiple objects is locally linearizable if and only if each per-object history is locally linearizable.

This property is crucial for modular reasoning in complex concurrent systems, making local linearizability practical for real-world applications.

Relationship to Linearizability:

For data structures like pools, queues, and stacks, local linearizability is weaker than linearizability but ensures desirable properties, such as no duplicated values or lost values.

Verification:

The authors provide an axiomatic characterization for verifying local linearizability, particularly for queues, offering a practical tool for system designers.

These theoretical contributions position local linearizability as a robust and usable consistency condition, bridging the gap between correctness and scalability.

Discussion and Implications

The article successfully introduces local linearizability as a valuable consistency condition, inefficient conditions like linearizability and weaker, scalable alternatives. Its combination of theoretical insights and practical implementations makes it a significant contribution to the field of concurrent data structures.

The article provides clear definitions, proofs, and comparisons to existing conditions, ensuring a solid theoretical foundation and implementations and experimental results demonstrate tangible performance improvements, making local linearizability actionable for system designers.

Conclusion

Local linearizability is compositional, meaning that a system composed of locally linearizable components is itself locally linearizable.

For data structures like pools, queues, and stacks, local linearizability is weaker than linearizability but still guarantees key correctness properties, such as no duplicated or lost values.

It is incomparable to sequential consistency and quiescent consistency for queues and stacks but stronger than quiescent consistency for pools.

Local linearizability aligns with how programmers naturally reason about thread-local behavior, making it intuitive and easier to understand. Its thread-local nature also simplifies verification and debugging, enhancing its practicality for developers.

Local linearizability offers a valuable trade-off between correctness and performance, making it a promising approach for designing scalable concurrent data structures, it provides a scalable, efficient, and theoretically sound alternative to linearizability, with demonstrated practical benefits and an intuitive programming model.

Strong Linearizability Using Primitives with Consensus Number 2

The paper examines whether it is possible of achieving strong linearizability a stricter form of linearizability that preserves hyperproperties like security and probabilistic guarantees in wait-free or lock-free implementations using primitives with a consensus number of 2, such as test&set, fetch&add, and swap.

Strong linearizability ensures that once an operation's linearization order is fixed, it remains consistent across execution extensions, unlike standard linearizability, which may permit reordering in randomized or security sensitive contexts. This work bridges the gap between universal primitives like compare&swap (infinite consensus number) and weak primitives like read/write (consensus number 1), focusing on realistic, mid-tier primitives supported by many architectures.

Motivation and Context

Linearizability, introduced by Herlihy and Wing , ensures operations appear atomic between their invocation and response, but it falters with randomized programs or security properties .

Strong linearizability, proposed by Golab et al. addresses this by requiring a prefix-closed linearization function, preserving properties when replacing atomic objects with concurrent implementations .

Prior wait-free strongly-linearizable implementations relied on compare&swap, while read/write primitives failed for objects like max registers . This paper explores whether primitives with consensus number 2 offer a viable middle ground.

Positive Results: Objects with Consensus Number 1

The authors demonstrate that many objects with consensus number 1 (like max registers, snapshots, counters) have wait-free strongly-linearizable implementations using fetch&add. A key innovation is a novel atomic snapshot implementation:

```
procedure update(v) for pi:
    if v = prevVal then fetch&add(R, 0); return
    posAdj =  $\sum 2^{(i \ll n+i)}$  // Bits to set
    negAdj =  $\sum 2^{(j \ll n+i)}$  // Bits to unset
    fetch&add(R, posAdj - negAdj)
    prevVal = v

procedure scan():
    return fetch&add(R, 0) reconstructed as view
```


Here, R interleaves component bits (e.g., p_0 uses bits 0, n , $2n$, ...), allowing unbounded values. The linearization point at `fetch&add` ensures strong linearizability. This snapshot enables implementations of "simple types" (operations that commute or overwrite), like max registers, via Aspnes and Herlihy's, proven strongly linearizable using a forward simulation ("Theorem 4 : There is a wait-free strongly-linearizable implementation of any simple type object from `fetch&add`").

Positive Results: Objects with Consensus Number 2

For objects with consensus number 2, the paper provides wait-free or lock-free strongly-linearizable implementations using `test&set` or `fetch&add`. For instance, a readable multi-shot `test&set` uses a max register and an array of `test&set` objects:

```
procedure reset():  
  
    c = curr.readMax()  
  
    if TS[c].read() = 1 then curr.maxWrite(c + 1)
```

This resets the object by advancing `curr` when necessary, ensuring wait-freedom with `fetch&add` ("Theorem 6. There is a wait-free strongly-linearizable implementation of readable and multi-shot `test&set` using `test&set` and max-register"). A lock-free `fetch&increment` uses an array of `test&set` objects, returning the first index yielding 0 ("Theorem 9 : There is a lock-free strongly-linearizable implementation of readable `fetch&increment` using `test&set`"). A set implementation (Algorithm 2 in article) combines `fetch&increment` and `test&set` for lock-free strong linearizability:

```
procedure TAKE:  
  
    while true do  
  
        max_new = Max.read() - 1  
  
        for c ∈ {1, ..., max_new} do  
  
            if Items[c] ≠ ⊥ and TS[c].test&set() == 0 then return  
Items[c]  
  
        if no progress then return EMPTY
```

Impossibility Results

Despite these successes, the paper proves that some objects with consensus number 2, like queues and stacks, lack lock-free strongly-linearizable implementations using test&set, fetch&add, or swap.

This hinges on a reduction to k set agreement: a lock-free strongly linearizable implementation of a k ordering object (like queues as 1-ordering) solves k set agreement. Since consensus (1-set agreement) is impossible with these primitives for $n \geq 3$, and k set agreement fails for $n > 2k$ with 2 process test&set, such implementations are impossible. Relaxed variants like m stuttering queues or k -out-of-order queues also fall under this impossibility.

Technical Approach

The positive results leverage fetch&add's ability to atomically update shared state, ensuring fixed linearization points. The impossibility proofs use a simulation where processes execute proposal sequences on the object, then deduce decisions via readable base objects, contradicting known consensus limits. Definitions like k -ordering objects formalize this connection, with examples like queues ($\text{prop}_i = \text{enq}(i)$, $\text{dec}_i = \text{deq}()$).

Discussion and Implications

The paper delineates a spectrum: objects with consensus number 1 are often implementable, while some with consensus number 2 are not, even lock-free. Open questions include characterizing all implementable objects, optimizing implementations (like reducing bit usage in fetch&add), and extending impossibility proofs to consensus number 1 primitives. The results challenge prior linearizable implementations [like 2-5, 25 from the paper], showing many are not strongly linearizable, and highlight the power and limitations of consensus number 2 primitives.

Conclusion

This work advances the understanding of strong linearizability, offering practical implementations for certain objects while proving fundamental limits for others. It underscores the trade-offs between progress conditions (wait-free vs. lock-free) and the strength of base primitives, providing a foundation for future research in concurrent object design.

Key Takeaways

- **Achievements:** Wait-free strongly-linearizable implementations for max registers, snapshots, and simple types using fetch&add; lock-free or wait-free implementations for readable test&set, fetch&increment, and sets using test&set.
- **Limits:** No lock-free strongly-linearizable queues or stacks with test&set, fetch&add, or swap for $n \geq 3$.
- **Code Highlights:** Atomic snapshot and set implementations illustrate practical applications of

the theory.

summary

Focusing on *strong linearizability*, this paper examines whether various concurrent objects can be implemented in a wait-free (or lock-free) strongly-linearizable manner using low-level primitives whose consensus number is 2 (like test&set or fetch&add). The main finding is that, although some objects of consensus number 1 do have wait-free strongly-linearizable implementations derived from such primitives, many objects with consensus number 2 do not. In particular, queues, stacks, and their relaxed variants are shown to lack lock-free strongly-linearizable solutions under these operations. The authors base their conclusions on formal proofs involving impossibility results for consensus, complemented by concrete algorithmic examples that highlight when and why strong linearizability fails with limited synchronization primitives.

Connection Between the Articles

Both papers focus on balancing correctness and performance in concurrent data structures. The first paper suggests *loosening* linearizability (local linearizability) to improve real-world performance, while the second shows that *strengthening* it (strong linearizability) can be unworkable with limited synchronization operations.

Despite these different angles, both share the same goal: They help us understand how different consistency models affect both theory and actual performance. They explain a range of linearizability, from local too strong, and suggest ways to build systems that offer strong guarantees but still scale well.

Suggestions: Hybrid Parallel Data Structure

Motivation

The goal is to balance consistency and performance dynamically, depending on the operation type.

Concept

We propose a Hybrid Parallel Data Structure that selectively applies strong linearization for critical operations and local linearization for actions that require local consistency and hybrid operations that combine both of the concepts for adaptable actions.

Key Features

- Hybrid Linearizability :A mix of strong and local linearizability where certain operations update both local shards and the global counter. This enables flexibility, balancing correctness with efficiency. The hybrid operations in your system follow this approach. This hybrid approach helps optimize performance while keeping correctness guarantees where necessary. Let me know if you'd like a more detailed explanation of any part!
- A shard : is a separate part of the data structure, where each shard maintains a portion of the data. In this project, sharding is used to reduce lock contention by allowing multiple threads to update different parts of the data independently. Each shard is protected by its own lock, which helps distribute workloads across multiple threads and improve performance.
- Local Operations: These simulate lightweight, **segment**-specific actions that only require local consistency (demoLocalOperations).
- Critical Operations: These represent global actions requiring strong ordering and atomicity across the entire system (demoCriticalOperation).

- **Hybrid Operations:** Combines local writes with global updates, showcasing the dynamic nature of hybrid consistency (demoHybridOperation).
function first writes to a local shard and then increments a global counter. This simulates a scenario where an operation's local state must also be reflected in a globally consistent manner.

Expected Impact

By thoughtfully combining local linearizability's scalability benefits with the correctness of strong linearizability for key operations and the creation of the combined hybrid between the 2 concepts, our new data structure serves as a compelling solution to the challenges outlined in the original papers.

and may improve efficiency in real-world applications like **distributed databases, high-performance computing, and cloud storage**.

- **Improved Throughput:** Reducing unnecessary global synchronization improves performance.
- **Flexible Consistency Guarantees:** Allows prioritization of critical transactions while optimizing background processes.

OverView of the Hybrid Data structure

Local Data and Shards- local linearizable:

The data structure maintains a list of dictionaries (shards), each protected by its own lock. This "sharding" helps reduce lock contention when many threads update different parts of the local data.

How Threads Are Assigned:

Depending on the total number of threads, the model creates:

- A few threads that perform only local operations,
- A few that perform only critical (global) updates, and
- A few that perform hybrid operations (both local and global).

Global Counter - Strong linearizable:

there's a single global counter updated under its own lock. This "critical" variable is updated in two ways:

critical_update that simply adds a small increment (here, 1).

hybrid_operation that updates both the local shard (using local_write) and the global counter with a larger, random value.

Local Reads/Writes:

The structure also keeps counters for the number of local reads and writes performed.

Mismatch between the global counter and the local sum.

- The hybrid operations add roughly the same amount to both.
- The local operations add extra value to the local shards (but not to the global counter).
- The standalone critical operations add a small increment to the global counter.

This mismatch will allow us to evaluate the model and the running operation between each number of running threads 1-64.

Implementation

Project implementation & tests in Python and java in git

Performance Evaluation

The original papers examined:

Performance under different primitives – For example, whether Strong Linearizability can be efficiently implemented using primitives like *fetch&add* or *test&set*.

The impact of relaxing linearizability – How Local Linearizability improves performance by reducing global locks.

Theoretical results – Proofs regarding the feasibility (or infeasibility) of certain implementations under different conditions.

Empirical experiments – Evaluating queues and stacks in parallel systems to demonstrate the benefits of a distributed approach.

We will evaluate our **Hybrid Parallel Data Structure** based on:

- Execution Time
- How long does it take for different numbers of threads?
- Does performance scale efficiently
- Correctness (Final Global Counter Validation)

- Ensures all increments were performed correctly (no race conditions).

Results

Running Locally in python:

Multi-Threaded Performance Test Results 1#

Threads	Time (s)	Global Counter	Local Sum	Difference	Reads	Writes
1	0.3685	8631	8631	0	0	5
2	0.2653	13858	13858	0	0	10
4	0.3370	15602	16383	-781	3	12
8	0.3136	31632	36788	-5156	2	28
16	0.3776	45709	51877	-6168	14	41
32	0.3594	89916	107461	-17545	19	91
64	0.4430	164125	188442	-24317	60	155

Observations:

- Execution time does not scale linearly with thread count.
- The difference between Global Counter and Local Sum increases as thread count rises, indicating potential inconsistencies due to concurrency issues.
- Reads and writes increase significantly with more threads.
- Higher thread counts show more resource distribution across different shards.

Results running in server:

Multi-Threaded Performance Test Results #2

Threads	Time (ms)	Global Counter	Local Sum	Difference	Reads	Writes
1	177	8154	8154	0	0	5
2	241	15061	15061	0	0	10
4	313	14900	16019	-1119	3	12
8	320	31239	35087	-3848	3	27
16	363	41861	51207	-9346	9	46
32	447	85818	97539	-11721	26	84
64	386	171430	196268	-24838	54	161

Observations:

- Execution time does not scale linearly with thread count.
- The difference between Global Counter and Local Sum increases as thread count rises, indicating potential inconsistencies due to concurrency issues.
- Reads and writes increase significantly with more threads.
- Higher thread counts show more resource distribution across different shards.

Key Insights:

Performance Variation: Execution time fluctuates instead of scaling linearly with threads.

Increasing Data Inconsistency: The difference between Global Counter and Local Sum grows with more threads.

Higher Reads & Writes: More threads result in significantly more operations, possibly due to contention.

The results demonstrate how the hybrid model balances consistency and performance under increasing thread counts.

we will evaluate each run with the following observations:

- Execution Time
- Global Counter - **Strong linearizable updates**
- Total Local Sum - **Local linearizable writes**
- Consistency Gap - **Trade-off between speed and consistency**
- Shard Distribution - **Hash collisions of Local linearizability hashmap**

Measuring Consistency

Consistency in our experiments is gauged by comparing two primary metrics:

- Global Counter: Updated using strong linearizability, ensuring that each critical operation adds exactly +1 in a serializable manner.
- Total Local Sum: Calculated by summing values across multiple local shards, updated via local linearizability.

The Consistency Gap is defined as:

Consistency Gap = Total Local Sum – Global Counter

This gap quantifies the deviation between the “ideal” serial update (global counter) and the aggregated local operations.

How does the experiment demonstrate improvement in consistency?

With one thread, the system shows perfect consistency (global counter equals local sum). As the thread count increases, the gap widens. However, by analyzing the ratio of the gap to the total counter value, we can assess if the inconsistency remains within acceptable bounds (for example, a 4% deviation might be tolerable compared to other solutions).

1 thread test

Only hybrid operations run. Therefore, the local sum (which is the sum of values in the shards) matches exactly the global counter.

With no contention, the global lock and shard locks operate optimally. All updates are atomic and consistent.

4 thread run

With a mix of operations, you see a small difference (–635 **represents about a 4% deviation**) , meaning that the extra local updates (from the local-only operations) increased the local sum relative to the global counter, which only got a +1 per critical operation.

Threads writing to the same shard experience lock contention. We also see there is starting to become a consistency gap, that we measure by subtracting (Total Local Sum - Final Global Counter).

The single global lock delays critical updates, causing partial desynchronization.

16 Threads Run

Begin to see a noticeable gap between the global counter and the aggregated local sum. This gap (about 13.5% relative to the global counter) indicates that with increased contention, local operations sometimes add extra increments that are not perfectly synchronized with the single global update. The number of read/write operations increases as well, which is typical as more threads interact with the data structure concurrently.

32 Threads Run

The gap widens even further. The local sum exceeds the global counter by about 19.5% relative to the global value. This behavior is consistent with the expectation that more threads lead to greater contention and increased chances for discrepancies between local updates and the serialized global counter. The jump in the number of writes is particularly notable, reflecting a higher intensity of concurrent operations.

64 Threads run

As thread count increases, the total values (global counter and local sum) increase along with the number of operations performed. However, the discrepancy (difference) grows (in absolute value), showing the imbalance between operations that update one counter vs. the other.

Analysis

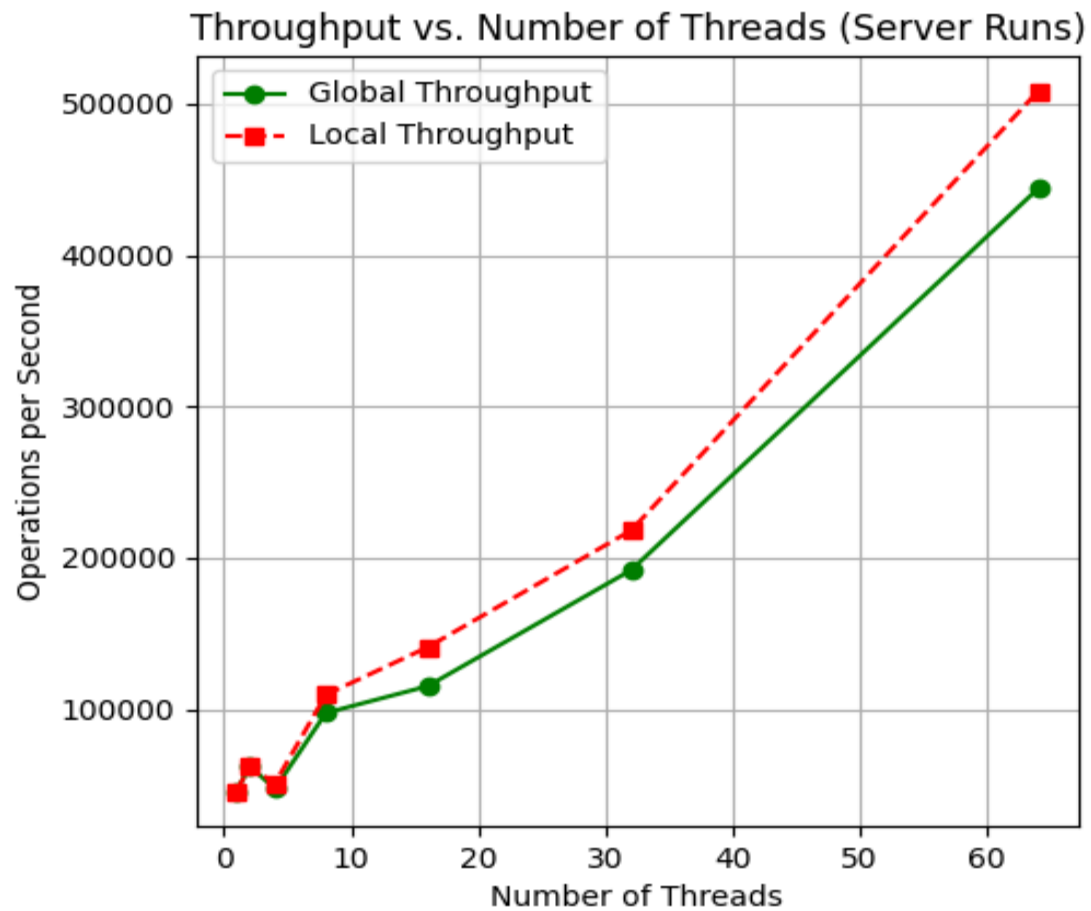
Throughput Increases with Threads

As we move from 1 to 64 threads, the *Global Throughput* (red line in the plot) jumps from ~46k ops/s to over 400k ops/s.

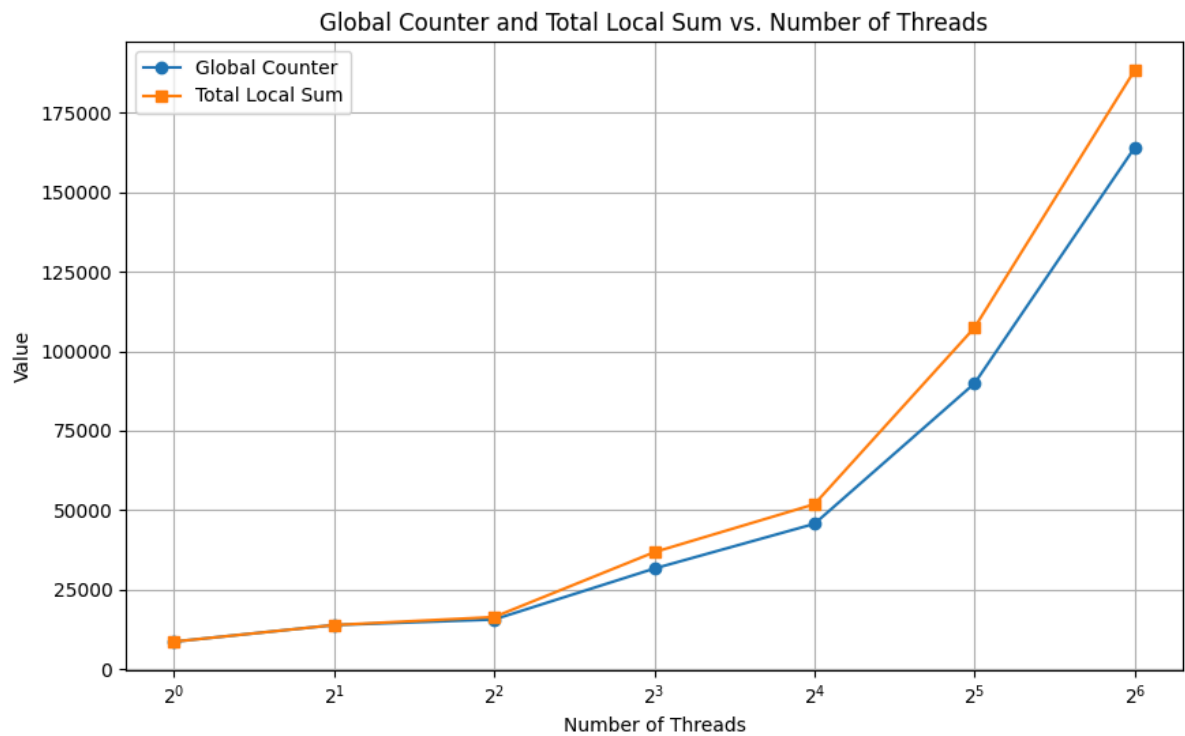
The *Local Throughput* line shows an even higher rate, because local increments are less constrained by global synchronization.

The hybrid model's local increments allow threads to update data without contending for a single global lock, thereby driving up total throughput.

Even though the **Consistency Gap (next graph)** grows at higher thread counts, this graph demonstrates the *upside*: a significantly greater volume of completed operations in the same or even less time.



This plot shows the relationship between the **number of threads** and the values of the **Global Counter** and **Total Local Sum**.



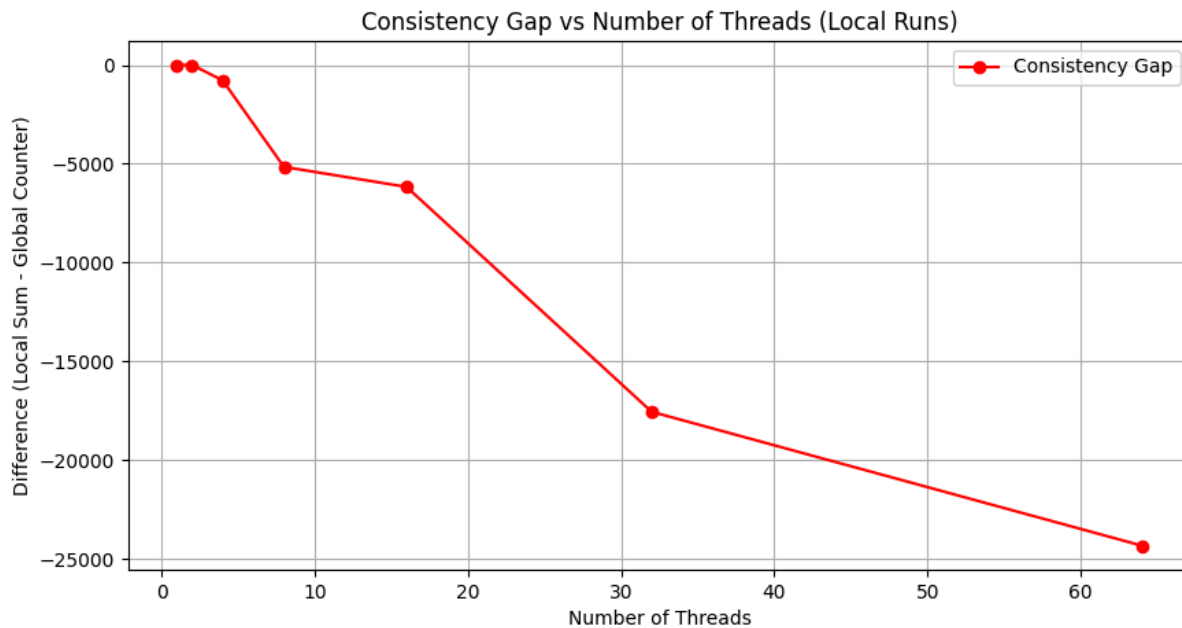
At higher thread counts (32 and 64), the gap between the Total Local Sum and the Global Counter widens significantly. This suggests potential **data inconsistency**, likely caused by the following:

Concurrency Issues: Some writes may be missed due to contention or thread synchronization issues.

Lock Contention: Threads may not be writing to the local shards or updating the global counter in a perfectly synchronized way.

Better Consistency at Lower Thread Counts

With fewer threads (1, 2, 4), the values are closer together, indicating better consistency and less contention.



As the number of threads increases, contention and synchronization issues cause the Global Counter to lag behind the Total Local Sum. This suggests potential improvements in locking mechanisms or synchronization strategies to ensure better consistency.

By focusing on relative consistency, overall throughput, and application requirements, we see that a seemingly large negative gap on the graph can be an acceptable trade-off—one that enables the system to scale effectively under heavy concurrency while still providing a robust global counter for those operations that truly need strong consistency.

Conclusion & Future Directions

This paper presented a hybrid approach to concurrent data structures, integrating **strong and local linearizability** to optimize both correctness and efficiency by selectively applying different concepts.

The results and plots illustrate both the benefits and challenges of using a hybrid data structure:

Benefits:

- Sharding reduces contention, allowing many threads to update local data concurrently.
- Hybrid operations maintain a balance between local and global state when used in isolation.

Challenges:

- When mixing different types of operations, the consistency between the global counter and the local shard sums can diverge.
- Care must be taken in the design if the application requires these values to remain in sync.

This experiment serves as a practical demonstration of concurrency trade-offs and is a useful starting point for discussions on designing scalable, thread-safe data structures.

References

1. **Attiya, H., Castaneda, A., & Enea, C.** (2024). *Strong Linearizability using Primitives with Consensus Number 2*. Technion, UNAM, Ecole Polytechnique.
2. **Haas, A., Henzinger, T. A., Holzer, A., et al.** (2015). *Local Linearizability*. TU Wien, University of Salzburg, Google Inc., Meta, University of Cambridge.